# Introduction to Recursion

Recursion is a process in which a problem is defined in terms of itself. It represents a technique in which a complex problem is solved by repeatedly splitting it into smaller sub-problems. The process continues until a specified stopping point is reached. The final solution (result) is obtained if and only if all sub-problems are solved and their results/solutions are combined to form the solution of the original problem.

To implement the recursive technique in programming, a function must be capable of calling itself. Therefore, recursion in programming is defined as a technique in which a function repeatedly calls itself, and the process stops when a base case condition is satisfied. Examples of such problems are Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc.

## Recursive Algorithm

An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with a smaller input.

## Recursively Defined Function

We can use the following method to define a function with the natural numbers as its domain:

- Specify the value of the function at zero.
- Give a rule for finding its value at any integer from its values at smaller integers.

Such a definition is called recursive or inductive definition.

## Recursive Function

A function is said to be recursive if it is allowed to call itself repeatedly and stops execution when the base case condition is satisfied. This repeated self-invocation of a function is called a recursive call. Each recursive call must move closer to the base case condition to ensure proper termination of the function.

## Principles/ Properties/ Components of Recursion

Following are two fundamental principles of recursion:

**Base Case/ Condition:** The base case/ condition is also known as the terminating, exit, or stopping condition. In recursion, when a keeps function calling itself repeatedly, program control must exit/ come out to avoid infinite execution. In such cases, the base case is required to break or stop the recursive calls at a specific point, thereby preventing infinite recursive calling. When the base case is satisfied, the function definition returns a direct value to the original caller. Without it, the function calls itself indefinitely, resulting in a stack overflow.

**Note:**

- ✓ When the base case is satisfied, the function does not call itself and recursion stops.
- ✓ When the base case is not satisfied, the function calls itself and recursion continues.

**Recursive Case:** The recursive case is the part where the method calls with a modified and simpler (smaller) form of the original input. Each recursive call moves closer to the base case.

**Key Concept**

In recursion, a complicated problem is solved by moving backward towards the base case/ condition by splitting it into smaller sub problem.

General Form of Recursive Function Structure

```
return_data_type func_name(list_of_data_type_of_args)
{
// base case
if (base_case)
 return base_value;
 else
 // recursive case
return method_name(modified_args);  // recursive call to func_name()
}
```

## Types of Recursion

Recursion can be classified into several types. The commonly used types of recursion are:

1. Direct Recursion
2. Indirect Recursion
3. Tail Recursion
4. Non-Tail Recursion
5. Linear Recursion
6. Tree Recursion

**Direct Recursion:** A function is said to have direct recursion when it calls itself directly from within its own body. This is the most common type.

### Use Cases:

- Factorial calculation
- Sum of natural numbers
- Power calculation

### Example:

```
#include <stdio.h>
int factorial(int n) {
   if (n == 0)  // Base case
      return 1;
   else
      return n * factorial(n - 1);  // Direct recursion
}
int main() {
   int num = 5;
   printf("Factorial of %d is %d", num, factorial(num));
   return 0;
}
```

**Output:**

Factorial of 5 is 120

**Time Complexity:** O(n)

**Space Complexity:** O(n)

**Indirect Recursion:** Indirect recursion occurs when two or more functions call each other, and one of them eventually calls the original function, forming a call cycle. It is also known as mutual recursion.

### Example:

```
#include <stdio.h>
int num = 1;
void odd() {
   if (num <= 10) {
      printf("%d ", num + 1);
      num++;
      even();  // Calls even function
   }
}
void even() {
```

```
    if (num <= 10) {
        printf("%d ", num - 1);
        num++;
        odd();  // Calls odd function
    }
}
int main() {
    odd();  // Initial function call
    return 0;
}
```

**Output:**

2 1 4 3 6 5 8 7 10 9

**Time Complexity:** O(n)

**Space Complexity:** O(n)

**Tail Recursion**: Tail recursion is a type of recursion in which the recursive call is the last operation performed by the function. This structure allows the compiler to optimize the recursion into an iterative process by reusing the same stack frame, which helps save memory and reduce the risk of stack overflow.

### Example:

```
#include <stdio.h>
void printNumbers(int n) {
    if (n == 0)
        return;  // Base case
    printf("%d ", n);  // Print the number
    printNumbers(n - 1);  // Tail recursive call
}
int main() {
    printNumbers(5);  // Prints numbers from 5 to 1
    return 0;
}
```

Output:

5 4 3 2 1

**Time Complexity:** O(n)

**Space Complexity:** O(1)

**Non-Tail Recursion:** There are operations to be performed after the recursive call returns, requiring the function to retain its stack frame. It also known as head recursion.

### Example:

```
#include <stdio.h>

void printNumbers(int n) {

   if (n > 0) {

      printNumbers(n - 1);  // Recursive call first

      printf("%d ", n);  // Prints after recursion

   }

}

int main() {

   printNumbers(5);  // Prints numbers from 1 to 5

   return 0;

}
```

**Output:**
1 2 3 4 5
**Time Complexity:** O(n)
**Space Complexity:** O(n)

**Linear Recursion**: Linear recursion occurs when a function calls itself once per execution, and the number of recursive calls grows linearly with the size of the input problem.

### Example:

```
#include <stdio.h>
int findMax(int arr[], int n) {
   if (n == 1)
      return arr[0];
   return (arr[n - 1] > findMax(arr, n - 1)) ? arr[n - 1] : findMax(arr, n - 1);  // Linear recursion
}
int main() {
   int arr[] = {4, 8, 15, 16, 23, 42};
```

```
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Maximum number is: %d", findMax(arr, n));
    return 0;
}
```

**Output:**

Maximum number is: 42

**Time Complexity: O(n)**

**Space Complexity: O(n)**

**Tree Recursion** Tree recursion occurs when a function makes more than one recursive call each time it is called, thus creating a branching structure. Examples of such recursive problems include a computation of Fibonacci numbers.

### Example:

```
#include <stdio.h>
int fibonacci(int n) {
    if (n <= 1)
        return n;
    return fibonacci(n - 1) + fibonacci(n - 2);  // Tree recursion
}
int main() {
    int n = 6;
    printf("Fibonacci number at position %d is: %d", n, fibonacci(n));
    return 0;
}
```

**Output:**

Fibonacci number at position 6 is: 8

**Time Complexity: O(2^n)**

**Space Complexity: O(n)**

### Applications in Data Structures

Recursion is a natural fit for problems that have a hierarchical or self-similar structure:

- Tree Traversals
- Graph Algorithms
- Sorting Algorithms
- Dynamic Programming and Backtracking
- Recursive Data Structures

**Advantages**

- Simple and clear code
- Good for divide-and-conquer problems
- Reduces code complexity
- Handles hierarchical data structures easily

**Disadvantages**

- Uses more stack memory
- Slower due to function calls
- Risk of stack overflow

| Iteration | Recursion |
|---|---|
| Iteration is a process of executing certain set of instructinos repeatedly, without calling the self function. | Recursion is a process of executing certain set of instructions repeatedly by calling the self function repeatedly. |
| The iterative functions are implemented with the help of for, while, do-while programming constructs. | Instead of making use of for, while, do-while the repeatation in code execution is obtained by calling the same function again and again over some condition. |
| The iterative methods are more efficient because of better execution speed. | The recursive methods are less efficient. |
| Memory utilization by iteration is less. | Memory utilization is more in recursive functions. |
| It is simple to implement. | Recursive methods are complex to implement. |
| The lines of code is more when we use iteration. | Recursive methods bring compactness in the program. |